3 - Hands On Lab

Write a systemd Service, Containerise with Docker, Verify Auto-Restart in < 5s

EdgeAI Engineering Bootcamp — Analog Data

What You Will Build

By the end of this lab you will have:

- A Python inference script packaged as a **systemd service** that starts at boot
- The same script **containerised with Docker** and managed by Docker Compose
- A verified proof that both the systemd service and the Docker container **restart automatically in under 5 seconds** after a crash

---

What You Need

- Raspberry Pi 4B or Pi 5 running Pi OS Bookworm (64-bit)
- SSH access to the Pi from your laptop
- Docker installed (Module 3.3 installation steps)
- A model file at `/home/pi/edgeai/models/model.tflite` *(If you don't have a model yet, the lab script simulates inference with `time.sleep` — any placeholder `.tflite` file will work)*

Part A — The Base Python Script

Before doing anything with systemd or Docker, you need a Python script to work with. Use this one for the lab — it simulates an inference loop and logs output clearly.

Step 1 — Create the project folder

```
mkdir -p /home/pi/edgeai/models
mkdir -p /home/pi/edgeai/logs
```

Step 2 — Create a placeholder model file

```
touch /home/pi/edgeai/models/model.tflite
```

Step 3 — Create the inference script

```
nano /home/pi/edgeai/inference.py
```

Paste this content:

```
#!/usr/bin/env python3
"""
Lab Inference Script — Module 3 Hands-on Lab
Analog Data EdgeAI Bootcamp

Simulates an Edge AI inference loop.
Designed to work as both a systemd service and a Docker container.
"""

import os
import sys
import time
import signal
import logging

# ── Configuration from environment ──────────────────────────
MODEL_PATH  = os.environ.get("MODEL_PATH", "/home/pi/edgeai/models/model.tflite")
LOG_LEVEL   = os.environ.get("LOG_LEVEL", "INFO")
INTERVAL    = float(os.environ.get("INFERENCE_INTERVAL", "2.0"))

# ── Logging setup ────────────────────────────────────────────
logging.basicConfig(
    level=getattr(logging, LOG_LEVEL.upper(), logging.INFO),
    format="%(asctime)s  %(levelname)-8s  %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)
log = logging.getLogger("edgeai-lab")

# ── Graceful shutdown ────────────────────────────────────────
def handle_shutdown(signum, frame):
    log.info("Shutdown signal received. Exiting cleanly.")
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_shutdown)
signal.signal(signal.SIGINT, handle_shutdown)

# ── Startup ──────────────────────────────────────────────────
log.info("=" * 55)
log.info("  Edge AI Inference Service — LAB VERSION")
log.info("=" * 55)
log.info(f"Model path : {MODEL_PATH}")
log.info(f"Interval   : {INTERVAL}s between frames")

if not os.path.exists(MODEL_PATH):
    log.critical(f"Model file not found at: {MODEL_PATH}")
    log.critical("Exiting. Fix the path and try again.")
    sys.exit(1)

log.info("Model file found. Starting inference loop.")
```

```
# ── Main loop ──────────────────────────────────────────────
frame_count = 0
start_time = time.time()

while True:
    frame_count += 1
    elapsed = time.time() - start_time
    fps = frame_count / elapsed if elapsed > 0 else 0

    log.info(f"Frame {frame_count:06d} | Elapsed: {elapsed:.1f}s | Avg FPS:
{fps:.2f}")

    # Simulate inference work
    time.sleep(INTERVAL)
```

Save and exit (`Ctrl + O` , Enter, `Ctrl + X`).

Step 4 — Test the script manually

Run it directly and confirm it works before adding systemd or Docker:

```
python3 /home/pi/edgeai/inference.py
```

Expected output:

```
2026-07-23 08:30:00  INFO
═══════════════════════════════════════════════════════
2026-07-23 08:30:00  INFO        Edge AI Inference Service — LAB VERSION
2026-07-23 08:30:00  INFO
═══════════════════════════════════════════════════════
2026-07-23 08:30:00  INFO      Model file found. Starting inference loop.
2026-07-23 08:30:00  INFO      Frame 000001 | Elapsed: 0.0s | Avg FPS: 0.00
2026-07-23 08:30:02  INFO      Frame 000002 | Elapsed: 2.0s | Avg FPS: 1.00
```

Press `Ctrl + C` to stop. If you see errors, fix them before moving on.

Part B — systemd Service

Step 1 — Create the `.env` file

```
nano /home/pi/edgeai/.env
```

```
MODEL_PATH=/home/pi/edgeai/models/model.tflite
LOG_LEVEL=INFO
INFERENCE_INTERVAL=2.0
```

Protect it:

```
chmod 600 /home/pi/edgeai/.env
```

Step 2 — Write the service file

```
sudo nano /etc/systemd/system/edgeai.service
```

Paste this:

```
[Unit]
Description=Edge AI Inference Service — Lab
After=network.target
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/edgeai
EnvironmentFile=/home/pi/edgeai/.env
ExecStartPre=/bin/bash -c 'test -f /home/pi/edgeai/models/model.tflite'
ExecStart=/usr/bin/python3 /home/pi/edgeai/inference.py
Restart=always
RestartSec=5
TimeoutStopSec=10
StandardOutput=journal
StandardError=journal
SyslogIdentifier=edgeai

[Install]
WantedBy=multi-user.target
```

Save and exit.

Step 3 — Enable and start the service

```
# Tell systemd about the new file
sudo systemctl daemon-reload

# Start it now
sudo systemctl start edgeai

# Check it is running
sudo systemctl status edgeai
```

You should see `Active: active (running)`. ✅

Step 4 — Watch the live logs

Open a second terminal (or a new tmux pane) and run:

```
journalctl -u edgeai -f
```

You should see the inference frames scrolling. Leave this running — you will need it for the crash test.

Step 5 — Enable auto-start at boot

```
sudo systemctl enable edgeai
```

Step 6 — 🧪 CRASH TEST — Verify auto-restart in under 5 seconds

This is the key verification step. You will deliberately kill the service and time how long it takes to restart.

In terminal 1 — note the time and kill the process:

```
# Find the process ID
sudo systemctl status edgeai | grep "Main PID"

# Kill it with SIGKILL (force kill — no graceful shutdown)
sudo kill -9 <PID>
```

Or in one command:

```
sudo kill -9 $(systemctl show -p MainPID edgeai | cut -d= -f2)
```

Watch terminal 2 (the `journalctl -f` window).

You should see the log output stop, then within 5 seconds, see the startup messages appear again:

```
Jul 23 08:35:00 raspberrypi systemd[1]: edgeai.service: Main process exited,
code=killed, status=9/KILL
Jul 23 08:35:00 raspberrypi systemd[1]: edgeai.service: Failed with result 'signal'.
Jul 23 08:35:05 raspberrypi systemd[1]: edgeai.service: Scheduled restart job,
restart counter is at 1.
Jul 23 08:35:05 raspberrypi systemd[1]: Started Edge AI Inference Service — Lab.
Jul 23 08:35:05 raspberrypi edgeai[3421]: Edge AI Inference Service — LAB VERSION
Jul 23 08:35:05 raspberrypi edgeai[3421]: Model file found. Starting inference
loop.
```

The gap between `Failed` and `Started` should be exactly 5 seconds (`RestartSec=5`). ✅

Record your result:

Test	Expected	Actual (fill in)
Time from crash to restart	≤ 5 seconds	___ seconds
Service status after restart	`active (running)`	___
Frame counter restarted from 1	Yes	___

Step 7 — Test boot persistence

```
sudo reboot
```

After the Pi restarts, SSH back in and check:

```
sudo systemctl status edgeai
```

It should be running without you doing anything. ✅

Part C — Dockerise the Inference Script

Now you will put the same script into a Docker container.

Step 1 — Create `requirements.txt`

```
nano /home/pi/edgeai/requirements.txt
```

```
# For the lab, we have no external pip packages
# Add packages here as you build the real app
# e.g. tflite-runtime, paho-mqtt, numpy
```

Even if it is empty, the file must exist for the Dockerfile to work.

Step 2 — Write the Dockerfile

```
nano /home/pi/edgeai/Dockerfile
```

```
# ──────────────────────────────────────────────────
# STAGE 1: builder
# Install any packages that need compilation
# ──────────────────────────────────────────────────
FROM python:3.11 AS builder
```

```
WORKDIR /build

# Copy requirements and install to a prefix directory
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt


# ─────────────────────────────────────────────────────
# STAGE 2: runtime
# Small, clean image — only what is needed to run
# ─────────────────────────────────────────────────────
FROM python:3.11-slim AS runtime

# Copy installed packages from builder
COPY --from=builder /install /usr/local

# Set working directory
WORKDIR /app

# Copy application source code
COPY inference.py .

# Create the models directory inside the container
RUN mkdir -p /app/models

# Set environment variable defaults
# These are overridden by docker-compose.yml or docker run -e
ENV MODEL_PATH=/app/models/model.tflite
ENV LOG_LEVEL=INFO
ENV INFERENCE_INTERVAL=2.0

# Run as non-root user
RUN useradd --create-home appuser && chown -R appuser /app
USER appuser

# Start the inference app
CMD ["python3", "inference.py"]
```

Step 3 — Write `.dockerignore`

```
nano /home/pi/edgeai/.dockerignore
```

```
.env
.git
__pycache__
*.pyc
*.log
venv/
```

```
logs/
backups/
```

Step 4 — Write `docker-compose.yml`

```
nano /home/pi/edgeai/docker-compose.yml
```

```
version: "3.9"

services:

  inference:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: edgeai_lab
    restart: always
    environment:
      - MODEL_PATH=/app/models/model.tflite
      - LOG_LEVEL=INFO
      - INFERENCE_INTERVAL=2.0
    volumes:
      - ./models:/app/models        # mount model files from Pi into container
    mem_limit: 256m
    cpus: "1.0"
```

Step 5 — Build the Docker image

From inside `/home/pi/edgeai/`:

```
cd /home/pi/edgeai
docker compose build
```

You will see Docker execute each step of the Dockerfile. First build takes a few minutes. Subsequent builds use the cache and are much faster.

Confirm the image was created:

```
docker images
# Should show edgeai_lab or similar with a recent timestamp
```

Step 6 — Start the container

```
docker compose up -d
```

Check it is running:

```
docker compose ps
```

Expected output:

```
NAME            IMAGE           COMMAND                  STATUS
edgeai_lab      edgeai-inference   "python3 inference.py"   Up 30 seconds
```

Step 7 — Watch the container logs live

```
docker compose logs -f inference
```

You should see the same inference output as when you ran the script directly. ✅

Step 8 — 🧪 CRASH TEST — Verify Docker auto-restart in under 5 seconds

In terminal 1 — find the container process ID and kill it:

```
# Get the container ID
docker ps

# Force kill the container
docker kill --signal=SIGKILL edgeai_lab
```

Watch terminal 2 (the `docker compose logs -f` window).

Docker should restart the container within a few seconds (Docker's default is much faster than systemd's 5 seconds — usually under 2 seconds).

You should see:

```
edgeai_lab exited with code 137
edgeai_lab  | Edge AI Inference Service — LAB VERSION
edgeai_lab  | Model file found. Starting inference loop.
edgeai_lab  | Frame 000001 | ...
```

Verify with:

```
docker compose ps
# Status should show "Up X seconds" confirming it restarted
```

Record your result:

Test	Expected	Actual (fill in)
Time from kill to restart	< 5 seconds	___ seconds
Container status after restart	`Up X seconds`	___
Frame counter restarted from 1	Yes	___

Step 9 — Stop the Docker service cleanly

```
docker compose down
```

Part D — Stop the systemd Service (Avoid Conflicts)

If you want to run both the systemd service and the Docker container, they both try to use the same Python script and same model — which is fine, but logs get mixed up.

For the remainder of the lab, stop one of them:

```
# If keeping Docker, stop systemd:
sudo systemctl stop edgeai
sudo systemctl disable edgeai

# If keeping systemd, stop Docker:
docker compose down
```

Lab Completion Checklist

Go through this list and tick off each item. Show it to your instructor.

Part A — Python Script

- [ ] I created `/home/pi/edgeai/inference.py` and it runs cleanly with `python3 inference.py`
- [ ] I understand what `os.environ.get()` does in the script

Part B — systemd Service

- [ ] I created and secured `/home/pi/edgeai/.env` with `chmod 600`
- [ ] I created `/etc/systemd/system/edgeai.service` with all correct fields
- [ ] I ran `daemon-reload`, `start`, `status` in the correct order
- [ ] `systemctl status edgeai` shows `Active: active (running)`

- [ ] I can see live logs with `journalctl -u edgeai -f`
- [ ] I ran `systemctl enable edgeai` for boot persistence
- [ ] **CRASH TEST PASSED** — Service restarted in ≤ 5 seconds after `kill -9`
- [ ] **REBOOT TEST PASSED** — Service was running after `sudo reboot` without manual intervention

Part C — Docker

- [ ] I created `Dockerfile` with two stages (builder and runtime)
- [ ] I created `.dockerignore`
- [ ] I created `docker-compose.yml` with `restart: always`, volumes, and resource limits
- [ ] `docker compose build` completed without errors
- [ ] `docker compose up -d` started the container
- [ ] `docker compose ps` shows the container as `Up`
- [ ] I can see live container logs with `docker compose logs -f inference`
- [ ] **CRASH TEST PASSED** — Container restarted automatically after `docker kill --signal=SIGKILL`

Extension Tasks (If You Finish Early)

These are optional but strongly encouraged:

1. **Add a second service to `docker-compose.yml`** — Add an Eclipse Mosquitto MQTT broker container and verify it starts alongside the inference container with `docker compose up -d`.
2. **Reduce the Docker image size** — Run `docker images` and note the current image size. Try to reduce it by adding more to `.dockerignore` and using a lighter base image. How small can you get it?
3. **Test `StartLimitBurst`** — Edit the systemd service file and set `RestartSec=1` and `StartLimitBurst=3`. Then make the Python script always crash immediately (add `sys.exit(1)` at the top). Watch how many times systemd retries before giving up. What does `systemctl status edgeai` show?
4. **Use `tmux` for the whole lab** — Repeat any part of the lab inside a `tmux` session. Practice detaching (`Ctrl+B d`), closing the SSH session completely, reconnecting, and reattaching to find everything still running.

Instructor Sign-Off

Check	Student Demo	Instructor Sign
systemd service running at boot		
Crash test — restart within 5s		
Docker container with resource limits		
Docker crash test — auto-restart		
